NLP Assignment 2

Petra Nicoara (S5972876), Sophia Lopotaru (s5916771), Teodora Dobreva (S5888441)

1 Data

1.1 The Dataset

The data used for training and testing of the models is AG's News Topic Classification Dataset. It is divided into training and testing datasets, containing 120000 and 7600 samples, respectively. Each dataset includes four classes with equal numbers of samples per class.
The dataset has three columns. The first one indicates the class index of the sample (a number from one to four, where '1' represents "World", '2' represents "Sports", '3' represents "Business", and '4' represents "Sci/Tech"). The second and third columns consist of the title and description of each sample, respectively. The dataset is balanced, as indicated on the Hugging Face website.

1.2 Split

The dataset is initially split into a training and a testing dataset. We further split the training dataset into data to use for training and development with a ratio of 9:1, respectively, using the `train_test_split()` function, from the `Sklearn` library. This resulted in a training dataset with 108000 samples and a validation dataset with 12000 samples.

1.3 Tokenization

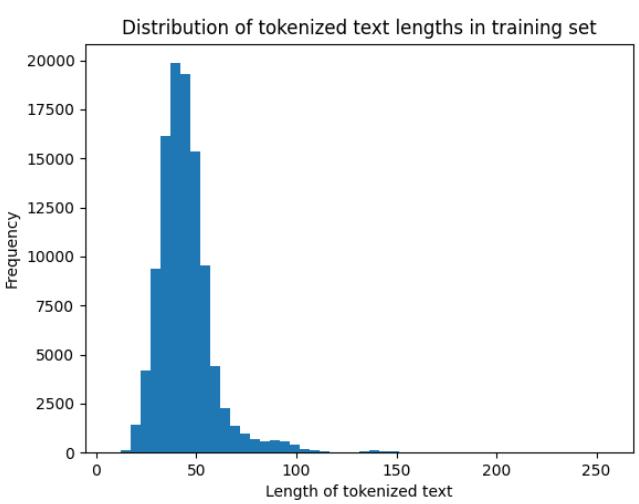

Figure 1: Distribution of lengths of the samples in the training dataset

For tokenization, we used the `UnicodeScriptTokenizer` from `tensorflow`. This tokenizer splits the words that are divided by white spaces and punctuation. This means that in cases where the words are followed by a punctuation mark, like at the end of a sentence, the word and the punctuation mark are obtained separately. Additionally, in cases such as contraction, the word is obtained separately from the apostrophe and the shortened word.
Based on the distribution of the sample lengths shown in Figure 1, we decided to set the maximum length of each sample to 64. In this way, we keep most of the samples with their original length and shorten the ones that have way longer sequences.
The maximum vocabulary length was set to 30000 with each word having a minimum of 2 occurrences in the training dataset.

2 Models

In order to implement both the Convolutional Neural Network (CNN) and the Long Short-Term Memory (LSTM) models, we used the `pytorch` library. We then performed hyperparameter tuning on `lr` (learning rate) and `embed_dim` (embedding dimension), using the function `do_hyperparameter_evaluation()`.
Both models are defined as classes and have `forward`, `evaluate` and `fit` function.
We implemented *early stopping* in our training loop. The `PATIENCE` value is 3, and the maximum number of epochs, `MAX_EPOCHS`, is set to 50.

2.1 CNN Text Classifier

The architecture of the CNN model can be observed in **Figure 2**: an **embedding layer**, a **dropout layer**, two **convolutional layers**, an **average pooling layer**, a **dropout layer**, a **linear layer**, and, finally, a **softmax layer**. The architecture is very similar to the implementation in the tutorial notebook.

The **Embedding Layer** (`nn.Embedding`) stores the embeddings and enables their retrieval using their indices.
The **Dropout Layer** (`nn.Dropout`) is used to regularize our model, by randomly setting some of the values in the input tensor to 0, with a probability of $p = 0.25$.
We then apply 2 **Convolutional Layers** (`nn.Conv1d`), with `kernel_sizes` $= (3, 4)$, to extract the main features.
We chose to add an **Average Pooling Layer** (`nn.AvgPool1d`), instead of a **Maximum Pooling Layer**, because this method extracts more general features. After that, there is another **Dropout Layer**, with the same purpose and setting as before ($p = 0.25$).

The number of `in_features` for the **Linear Layer** was calculated using **Formula 1**, where `output_size` represents the dimension of the output of the **Average Pooling Layer**.

$$output_size = \frac{input_size - kernel_size + 2 * padding}{stride} + 1 \quad (1)$$

We set the `kernel_size` to 3, `stride` to 1, and the number of kernels is set to 2. The `padding` is 0. For this reason, the final equation is as seen in **Formula 2**.

$$\texttt{in_features} = \texttt{num_filters} * \texttt{len(kernel_sizes) - 2} \quad (2)$$

The number of `out_features` of the **Linear Layer** is 4, which corresponds to the number of classes we are performing the classification for.

Finally, a **Softmax Layer** is applied to convert the output of the previous layer into a probability distribution, in the range $[0, 1]$, with all values summing up to 1. This is the final step for the classification.

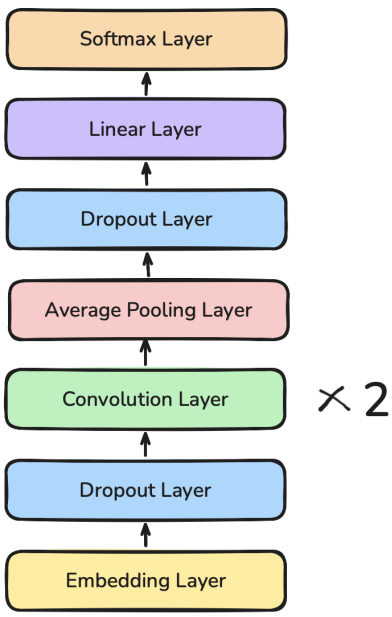

Figure 2: Depiction of the implemented CNN architecture. The data flows up from the bottom (i.e. from **Embedding Layer** to **Softmax Layer**).

2.2 LSTM Classifier

The architecture of the LSTM model is presented in **Figure 3**: an **embedding layer**, a **dropout layer**, an **LSTM layer**, an **average pooling layer**, a **dropout layer**, a **linear layer**, and last a **softmax layer**.

The **Embedding Layer** (`nn.Embedding`) works the same as the CNN embedding layer, where it stores embeddings and enables their retrieval through token indices.

The **Dropout Layer** (`nn.Dropout`) regularizes the model in order to reduce overfitting and improve the model's ability to generalize. It achieves this by randomly setting some of the values in the embedding tensor to zero with the probability of $p = 0.3$.

Afterwards, the sequence of embeddings is passed to the **LSTM Layer** (`nn.LSTM`) with two layers, which reads the input sequence, then learns the relationships between the elements and encodes the information in the hidden states, which represent contextual information. The hidden state from the final LSTM layer is used as the representation of the whole input sequence.

The **Average Pooling Layer** (`nn.AvgPool1d`) is used to create a smaller and smoother representation of the feature vector by computing the mean value of smaller groups from the feature vector.

Another **Dropout Layer** with the probability $p = 0.3$ is applied to again reduce the risk of overfitting.

The **Linear Layer** (`nn.Linear`) applies a linear transformation to the extracted features, creating a vector with the size equal to the number of classes.

Lastly, the **Softmax Layer** (`nn.Softmax`) converts the output of the **Linear Layer** into a probability distribution, in the range of [0, 1] which sum up to 1. This allows the model to choose the class with the highest probability as its final prediction.

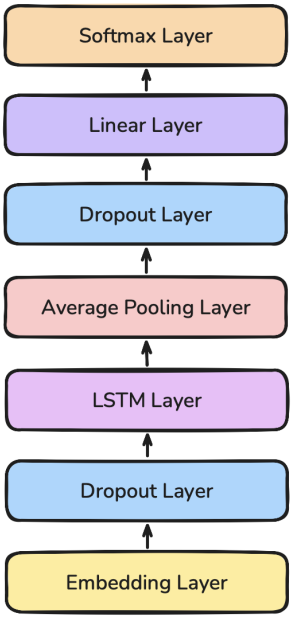

Figure 3: Depiction of the implemented LSTM architecture. The data flows up from the bottom (i.e. from **Embedding Layer** to **Softmax Layer**).

3 Results

3.1 Hyperparameter Tuning

We performed hyperparameter tuning on the `lr` and `embed_dim` parameters which correspond to the learning rate of the model and the embedding dimensions of the embedding layer. We trained models with various combinations of these parameters, on both 500 samples in the dataset (see **Figure**

4, and on the entire dataset (see **Figure 5**. We can observe that the combination of a learning rate of 0.001 and 64 embedding dimensions is the one with the highest score in both cases. Thus, those are the metrics we used in the training and initialization of the models.
It is interesting to see that we got the same results performing the hyperparameter tuning on such a small subset and on the whole dataset (those could have been 11 hours of training easily avoidable – yes, we probably should have done more optimization, but we learned our lesson).

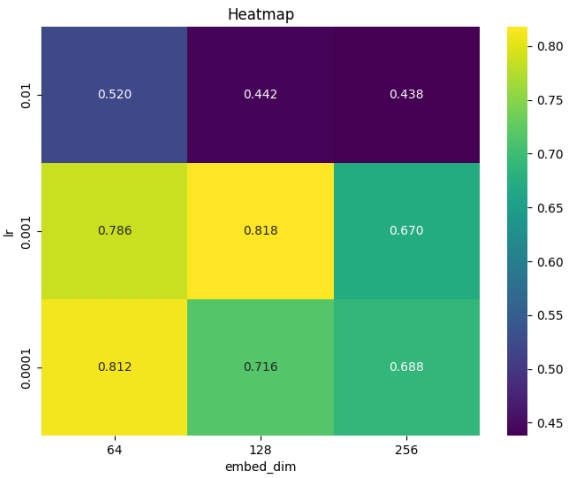

Figure 4: A heatmap showing the accuracy of each combination of hyperparameters used in the tuning process, when the models are trained on 500 samples.

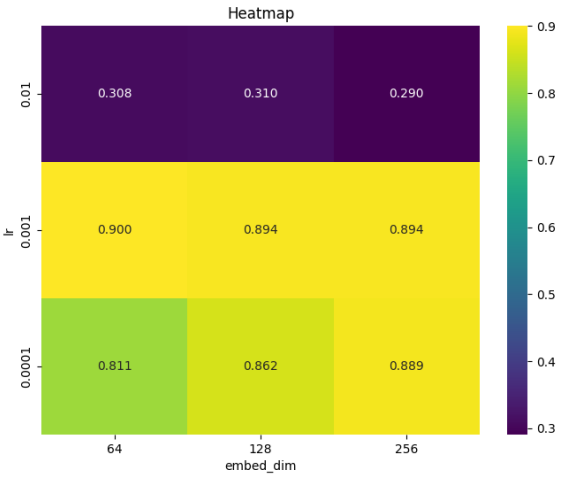

Figure 5: A heatmap showing the accuracy of each combination of hyperparameters used in the tuning process, when the models are trained on the entire dataset.

3.2 Evaluation on Validation Dataset

3.2.1 Accuracy metrics

We obtain an accuracy of approximately 89% for the CNN model and an accuracy of approximately 86% for the LSTM model on the validation set. The `f1-scores` are approximately 89% for the CNN model and approximately 86% for the LSTM model on the validation set. The values for the accuracy and `f1-score` are very similar because the dataset is balanced.
Overall, these results tell us that the models perform fairly well on the validation set, with the CNN model being slightly more accurate than the LSTM one.

3.2.2 Confusion matrices

Figure 6 represents the confusion matrix of the CNN model, while **Figure 7** represents the confusion matrix of the LSTM model. These matrices are indicators of the performance of the models on the validation set.
It can be observed from **Figure 6** that the CNN model classifies the best texts belonging to class 2 ("Sports"), while texts belonging to class 3 ("Business") are the hardest to classify.
Figure 7 shows that the LSTM model classifies the best texts belonging to class 2 ("Sports"), and that it is equally poor at classifying texts belonging to class 1 ("World") and to class 3 ("Business").
It can also be observed that a lot of texts belonging to class 3 ("Business") are misclassified as class 4 ("Sci/Tech") for both models. Similarly, a lot of texts belonging to class 1 ("World") and to class 4 ("Sci/Tech") are misclassified as class 3 ("Business"), especially by the LSTM model.
It is interesting to see that both models are good at classifying texts belonging to class 2 ("Sports"). That is probably because of the vocabulary and the uniqueness of the words that correspond to the label.

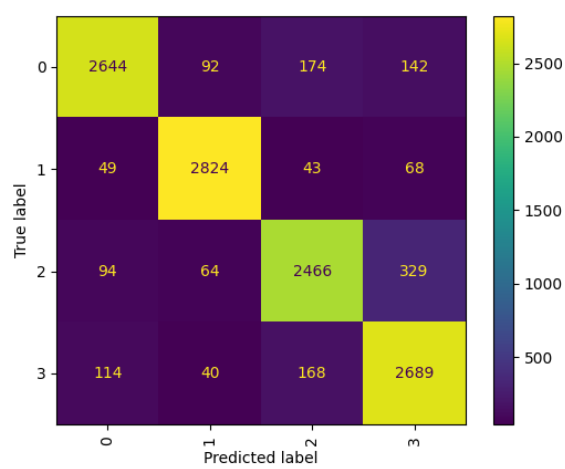

Figure 6: Confusion matrix from the CNN classifier evaluation on the validation set. The y-axis represents the true label, while the x-axis represents the predicted label. Labels 0, 1, 2, 3 correspond to classes 1 ("World"), 2 ("Sports"), 3 ("Business"), and 4 ("Sci/Tech"), respectively.

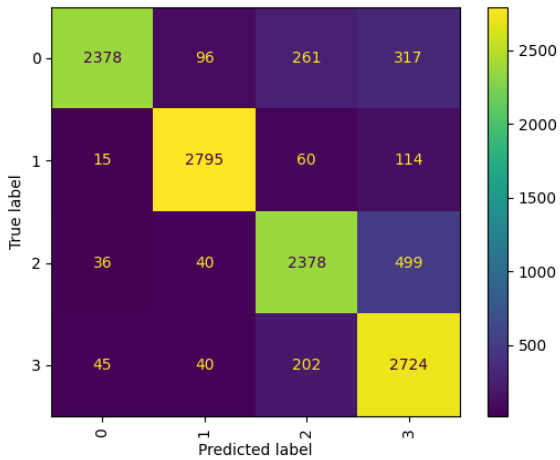

Figure 7: Confusion matrix from the LSTM classifier evaluation on the validation set. The y-axis represents the true label, while the x-axis represents the predicted label. Labels 0, 1, 2, 3 correspond to classes 1 ("World"), 2 ("Sports"), 3 ("Business"), and 4 ("Sci/Tech"), respectively.

3.2.3 Leaning curves

The learning curves of the loss and Macro F1 scores for both models are shown in **Figure 8** and **Figure 9**, respectively. We can see that the F1 score for the CNN gradually increases while the loss gradually decreases, while the F1 score of the LSTM stops improving around 85% while the loss keeps going down. This indicates there is no significant overfitting or underfitting of the model. The plateau of the loss of the CNN model suggests that it might be underfitting to the data. However, the F1 score of the same model keeps improving while the loss stays relatively the same, meaning it does not cause significant harm to the model's performance.
Overall, it looks like the early stopping does a good job at stopping the training before the models start overfitting the training data.

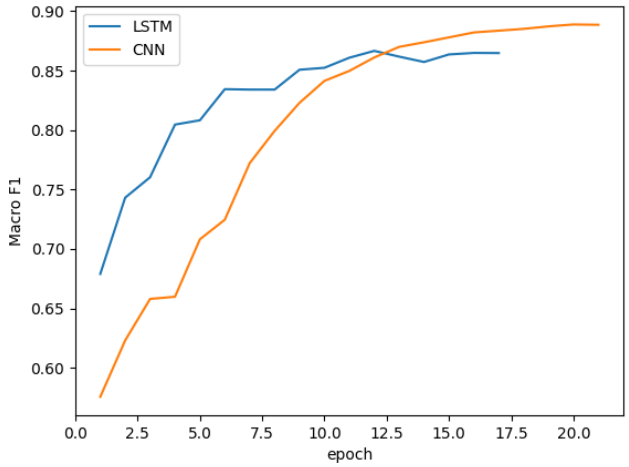

Figure 8: Learning curves of the Macro F1 score for the LSTM (blue) and CNN(orange)

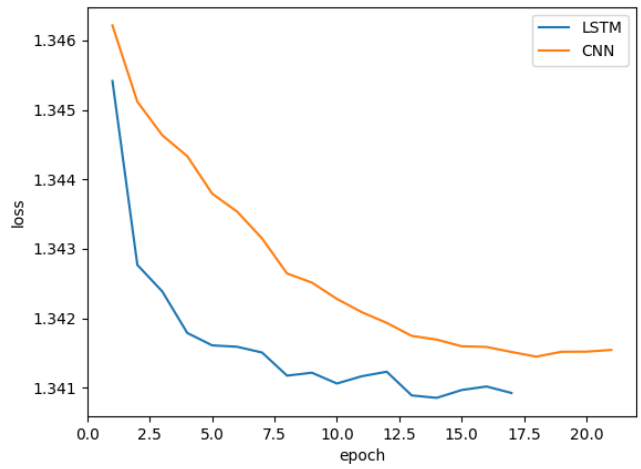

Figure 9: Learning curves of the loss score for the LSTM (blue) and CNN(orange)

3.3 Evaluation on Testing Dataset

3.3.1 Accuracy metrics

We obtain an accuracy of approximately 88% for the CNN model and an accuracy of approximately 85% for the LSTM model on the test set. The `f1-scores` are approximately 88% for the CNN model and approximately 85% for the LSTM model on the test set. The values for the accuracy and `f1-score` are very similar because the dataset is balanced.
Overall, these results tell us that the models perform fairly well on the test set, with the CNN model being slightly more accurate than the LSTM one.

3.3.2 Confusion matrices

Figure 10 represents the confusion matrix of the CNN model, while **Figure 11** represents the confusion matrix of the LSTM model. These matrices are indicators of the performance of the models on the test set.
It can be observed from **Figure 10** that the CNN model classifies the best texts belonging to class 2 ("Sports"), while texts belonging to class 3 ("Business") are the hardest to classify.
Figure 11 shows that the LSTM model classifies the best texts belonging to class 2 ("Sports"), while texts belonging to class 1 ("World") are the hardest to classify.
It can also be observed that a lot of texts belonging to class 3 ("Business") are misclassified as class 4 ("Sci/Tech").
It can also be observed that a lot of texts belonging to class 3 ("Business") are misclassified as class 4 ("Sci/Tech") for both models. Similarly, a lot of texts belonging to class 1 ("World") and to class 4 ("Sci/Tech") are misclassified as class 3 ("Business"), especially by the LSTM model.
It is interesting to see that both models are good at classifying texts belonging to class 2 ("Sports").

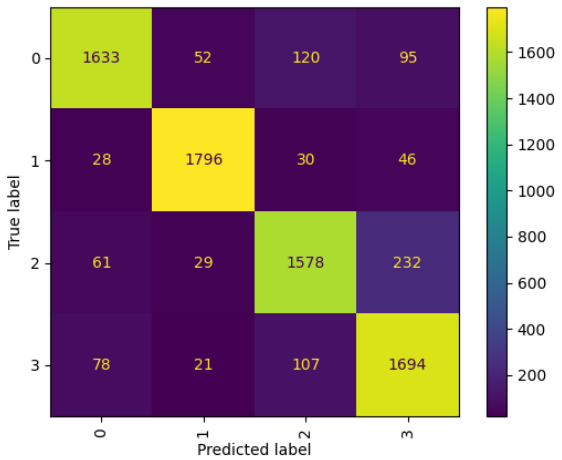

Figure 10: Confusion matrix from the CNN classifier evaluation on the testing set. The y-axis represents the true label, while the x-axis represents the predicted label. Labels 0, 1, 2, 3 correspond to classes 1 ("World"), 2 ("Sports"), 3 ("Business"), and 4 ("Sci/Tech"), respectively.

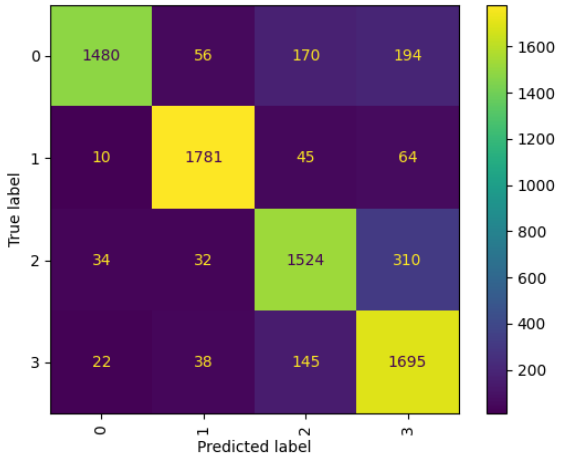

Figure 11: Confusion matrix from the LSTM classifier evaluation on the testing set. The y-axis represents the true label, while the x-axis represents the predicted label. Labels 0, 1, 2, 3 correspond to classes 1 ("World"), 2 ("Sports"), 3 ("Business"), and 4 ("Sci/Tech"), respectively.

4 Ablation

The ablation study was performed on the regularization rate of the dropout layer. Originally, the model was trained with a regularization rate equal to 0.3. To find what the effect of the dropout layers is on the final results, we set the regularization rate of a model (with the same metrics otherwise) to 0 and trained it with the same training and validation datasets.
The results of the models can be seen in **Table 1**. The LSTM classifier with dropout set to 0 has an accuracy score and a Macro-F1 of nearly 85%, while the LSTM classifier with dropout set to 0.3 has an accuracy score and a Macro-F1 of around 87%.
The CNN classifier with dropout set to 0 has an accuracy score and a Macro-F1 of 89%, while the LSTM classifier with dropout set to 0.3 has an accuracy score and a Macro-F1 of around 89.6%.
In both cases (when doing the ablation study on the LSTM and on the CNN), the accuracy of the models with a dropout of 0 on the training set was higher compared to the accuracy on the validation set.
In both cases, the accuracy of the models with a dropout of 0.3 on the training set was the same as that on the validation set.
Those results suggest that the models with dropout set to 0 overfit more to the data compared to the models with dropout set to 0.3. Since the dropout layers are used for regularization of the models, these results were expected.

Model	dropout	accuracy on training set	accuracy on validation set	F1 score
LSTM	0	91%	84.7%	84.7%
LSTM	0.3	87%	86.9%	87%
CNN	0	92.3%	89.1%	89%
CNN	0.3	89.5%	89.6%	89.6%

Table 1: Results from the ablation study on the effect of the dropout rate

5 Error Analysis

To perform error analysis, we collected 10 mistakes from each model, which turned out to be identical. **Table 2** in Appendix A shows the true labels, the predicted labels, and the text of the obtained mistakes.
Most of the errors we obtained have a true label "Sci/Tech" and a predicted label "World". This suggests that the vocabulary might be lacking words specific to the Sci/Tech articles, or that there are very few words that those articles share with each other (and maybe have a lot of common words with the "World" articles). Given the points we covered in **Section 6**, it is very likely the former is the case.

Most of the obtained mistakes contain company names and other words that might not be very commonly used. Usually, knowing that a particular name is always associated with, for example, business might be useful information. However, we do not obtain such information in the preprocessing. Thus, even after solving the problems from **Section 6**, samples like these might still get misclassified due to the words in them not being common enough to be included in the vocabulary and associated with a particular class.

5.1 Relation to Error Analysis in Assignment 1

In assignment 1, it seemed that the models made a lot of mistakes based on specific jargon connected to a particular class that was used in an article of a different class. For example, in a "World" article, there was a description of running disci-

plines from the Olympics in Greece and the model classified that sample as part of class "Sport".
However, the models for this assignment most likely make mistakes because of the quality and/or size of the vocabulary.

6 Critical Assessment

We would like to point out a few things that could have been implemented better. However, we did not have enough time to implement them (after we realized it would be good to) and then run the code to obtain the results and analyze them.
Firstly, we did not put the text data in all lowercase, which means some of the words in the vocabulary might be the same, but in a different form, which we think should have been avoided.
Additionally, we did the hyperparameter tuning on only one of the models. Initially, we thought that if we were going to use the same metrics for both models, we didn't need to include both of them in the hyperparameter tuning. However, the two model classes are different, and it might have been a better idea to make the heatmap with the average score for both model classes, or to make two separate heatmaps. In this way, we can choose the tunable hyperparameters so that none of the models has an advantage over the other.
Moreover, the vocabulary also includes some punctuation marks, which are not helpful to the learning of the model, since punctuation does not differ between news categories. Thus, not including punctuation from the vocabulary might have yielded better results.

7 Reproducibility

The results shown in this report were obtained with a random seed of 67.
The results can be replicated by using the following command in the terminal:
`python main.py`.

Appendix A

№	Predicted Label	True label	Text
1	World	Business	Fears for T N pension after talksUnions representing workers at Turner Newall say they are 'disappointed' after talks with stricken parent firm Federal Mogul.
2	World	Sci/Tech	The Race is On: Second Private Team Sets Launch Date for Human Spaceflight (SPACE.com)SPACE.com - TORONTO, Canada – A second\team of rocketeers competing for the #36;10 million Ansari X Prize, a contest for\privately funded suborbital space flight,...
3	World	Sci/Tech	Ky. Company Wins Grant to Study Peptides (AP)AP - A company founded by a chemistry researcher at the University of Louisville won a grant to develop a method of producing better peptides, which are short chains of amino acids, the building blocks of ...
4	World	Sci/Tech	Prediction Unit Helps Forecast Wildfires (AP)AP - It's barely dawn when Mike Fitzpatrick starts his shift with a blur of colorful maps, figures and endless charts, but already he knows what the day will bring. Lightning will strike in places he expec...
5	World	Sci/Tech	Calif. Aims to Limit Farm-Related Smog (AP)AP - Southern California's smog-fighting agency went after emissions of the bovine variety Friday, adopting the nation's first rules to reduce air pollution from dairy cow manure.
6	World	Sci/Tech	Open Letter Against British Copyright Indoctrination in SchoolsThe British Department for Education and Skills (DfES) recently launched a "Music Manifesto" campaign, with the ostensible intention of educating the next generation of British musicians....
7	World	Sci/Tech	Loosing the War on Terrorism\\"Sven Jaschan, self-confessed author of the Netsky and Sasser viruses, is\responsible for 70 percent of virus infections in 2004, according to a six-month\virus roundup published Wednesday by antivirus company Sophos."\\...
8	World	Sci/Tech	FOAFKey: FOAF, PGP, Key Distribution, and Bloom Filters\\}FOAF/LOAF and bloom filters have a lot of interesting properties for social\network and whitelist distribution.\\I think we can go one level higher though and include GPG/OpenPGP key\fingerpri...
9	World	Sci/Tech	E-mail scam targets police chiefWiltshire Police warns about "phishing" after its fraud squad chief was targeted.
10	World	Sci/Tech	Card fraud unit nets 36,000 cardsIn its first two years, the UK's dedicated card fraud unit, has recovered 36,000 stolen cards and 171 arrests - and estimates it saved 65m.

Table 2: 10 mistakes from the LSTM and the CNN classifiers